Documentation System

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Sage Doc Builders 1

2 Sphinx Extensions 29

Python Module Index 45

Index 47

CHAPTER

ONE

SAGE DOC BUILDERS

1.1 Sage docbuild main

This module defines the Sage documentation build command:

```
sage --docbuild [OPTIONS] DOCUMENT (FORMAT | COMMAND)
```

If `FORMAT` is given, it builds `DOCUMENT` in `FORMAT`. If `COMMAND` is given, it returns information about `DOCUMENT`.

Run `sage --docbuild` to get detailed explanations about arguments and options.

Positional arguments:

```
DOCUMENT              name of the document to build. It can be either one of
                      the documents listed by -D or 'file=/path/to/FILE' to
                      build documentation for this specific file.
FORMAT or COMMAND     document output format (or command)
```

Standard options:

```
-h, --help            show a help message and exit
-H, --help-all        show an extended help message and exit
-D, --documents       list all available DOCUMENTs
-F, --formats         list all output FORMATs
-C DOC, --commands DOC
                      list all COMMANDs for DOCUMENT DOC; use 'all' to list all
-i, --inherited       include inherited members in reference manual; may be
                      slow, may fail for PDF output
-u, --underscore      include variables prefixed with '_' in reference
                      manual; may be slow, may fail for PDF output
-j, --mathjax, --jsmath
                      ignored for backwards compatibility
--no-plot             do not include graphics auto-generated using the '.. plot'␣
↪markup
--include-tests-blocks
                      include TESTS blocks in the reference manual
--no-pdf-links        do not include PDF links in DOCUMENT 'website';
                      FORMATs: html, json, pickle, web
--live-doc            make Sage code blocks live for html FORMAT
--warn-links          issue a warning whenever a link is not properly
                      resolved; passes '-n' (nitpicky) to sphinx, except
                      for first-pass (inventory) builds
```

(continues on next page)

(continued from previous page)

```
--check-nested        check picklability of nested classes in DOCUMENT 'reference'
--no-prune-empty-dirs
                      do not prune empty directories in the documentation source
--use-cdns            assume internet connection and use CDNs; in particular,
                      use MathJax CDN
-N, --no-colors       do not color output; does not affect children
-q, --quiet           work quietly; same as --verbose=0
-v LEVEL, --verbose LEVEL
                      report progress at LEVEL=0 (quiet), 1 (normal), 2
                      (info), or 3 (debug); does not affect children
-o DIR, --output DIR  if DOCUMENT is a single file ('file=...'), write output
                      to this directory
```

Advanced options:

```
Use these options with care.

-S OPTS, --sphinx-opts OPTS
                      pass comma-separated OPTS to sphinx-build; must precede
                      OPTS with '=', as in '-S=-q,-aE' or '-S="-q,-aE"'
-U, --update-mtimes   before building reference manual, update modification
                      times for auto-generated reST files
-k, --keep-going      Do not abort on errors but continue as much as possible
                      after an error
--all-documents ARG   if ARG is 'reference', list all subdocuments of
                      en/reference. If ARG is 'all', list all main documents
```

class sage_docbuild.__main__.**IntersphinxCache**

Bases: object

Replace sphinx.ext.intersphinx.fetch_inventory by an in-memory cached version.

fetch_inventory(*app*, *uri*, *inv*)

Return the result of `sphinx.ext.intersphinx.fetch_inventory()` from a cache if possible. Otherwise, call `sphinx.ext.intersphinx.fetch_inventory()` and cache the result.

sage_docbuild.__main__.**command_line_args**()

Return the arguments that the builder is to parse.

Those of SAGE_DOCBUILD_OPTS come first, so that the ones written on the command line override them.

sage_docbuild.__main__.**format_columns**(*lst*, *align='<'*, *cols=None*, *indent=4*, *pad=3*, *width=80*)

Utility function that formats a list as a simple table and returns a Unicode string representation.

The number of columns is computed from the other options, unless it's passed as a keyword argument. For help on Python's string formatter, see

https://docs.python.org/library/string.html#format-string-syntax

A source tree holding no document at all is listed as the empty table it is, rather than making the help fail.

sage_docbuild.__main__.**generate_doc_sources**(*source_dir*)

Write the documentation sources that the repository does not keep.

The installation instructions, the page of every package and the options of the command line are generated from the package metadata and from the parser itself. A distribution ships them and meson writes them into its build

directory, but a checkout has neither until something makes them, and Sphinx would only report an include that it cannot read.

Which files the generator writes depends on the package metadata, so it is asked itself whether the tree is complete rather than probed for a file or two: a tree missing, or holding a truncated or an outdated copy of, any one of them builds into an error.

Failing to write them is an error too. The build that follows would report an include that it cannot read, or document the packages of a Sage other than the one asked for, and neither says what went wrong.

sage_docbuild.__main__.**get_formats**()

Return a list of output formats the Sage documentation builder will accept on the command-line.

The formats are those of any builder, so this needs no document: building one here to ask it would need the options that are still being parsed.

sage_docbuild.__main__.**help_commands**(*name='all'*)

Append and return a tabular list of commands, if any, the Sage documentation builder can run on the indicated document. The default is to list all commands for all documents.

sage_docbuild.__main__.**help_description**(*s=''*, *compact=False*)

Append and return a brief description of the Sage documentation builder.

If 'compact' is `False`, the function adds a final newline character.

sage_docbuild.__main__.**help_documents**()

Append and return a tabular list of documents available to the Sage documentation builder.

sage_docbuild.__main__.**help_examples**(*s=''*)

Append and return some usage examples for the Sage documentation builder.

sage_docbuild.__main__.**help_formats**()

Append and return a tabular list of output formats available to the Sage documentation builder.

class sage_docbuild.__main__.**help_message_long**(*option_strings*, *dest*, *nargs=None*, *const=None*, *default=None*, *type=None*, *choices=None*, *required=False*, *help=None*, *metavar=None*, *deprecated=False*)

Bases: `Action`

Print an extended help message for the Sage documentation builder and exits.

class sage_docbuild.__main__.**help_message_short**(*option_strings*, *dest*, *nargs=None*, *const=None*, *default=None*, *type=None*, *choices=None*, *required=False*, *help=None*, *metavar=None*, *deprecated=False*)

Bases: `Action`

Print a help message for the Sage documentation builder.

The message includes command-line usage and a list of options. The message is printed only on the first call. If error is True during this call, the message is printed only if the user hasn't requested a list (e.g., documents, formats, commands).

sage_docbuild.__main__.**help_usage**(*s=''*, *compact=False*)

Append and return a brief usage message for the Sage documentation builder.

If 'compact' is False, the function adds a final newline character.

class sage_docbuild.__main__.**help_wrapper**(*option_strings*, *dest*, *nargs=None*, *const=None*, *default=None*, *type=None*, *choices=None*, *required=False*, *help=None*, *metavar=None*, *deprecated=False*)

Bases: Action

A helper wrapper for command-line options to the Sage documentation builder that print lists, such as document names, formats, and document-specific commands.

sage_docbuild.__main__.**main**()

sage_docbuild.__main__.**sage_root_for**(*source_dir*)

Return a Sage source tree able to generate documentation sources, or None.

A distribution ships the generated sources and has no such tree.

The tree holding source_dir comes first: the sources of a second checkout are generated from the package metadata of *that* checkout, not from the one that happens to be running.

Every directory on the way up is tried, first of the path with its symbolic links resolved and then of the path as it was given. The resolved tree has to win when a link kept below one checkout points into another one. <root>/src/doc is the usual place, but meson writes into <root>/build/<tag>/src/doc.

sage_docbuild.__main__.**setup_logger**(*verbose=1*, *color=True*)

Set up a Python Logger instance for the Sage documentation builder.

The optional argument sets logger's level and message format.

sage_docbuild.__main__.**setup_parser**()

Set up and return a command-line ArgumentParser instance for the Sage documentation builder.

sage_docbuild.__main__.**source_dir_for_help**(*argv=None*)

Return the documentation sources that the lists printed by the help are read from.

The help is printed while the command line is still being parsed, so a --source that follows the option asking for the list has not reached the parser yet: it is read from argv here, whatever its place, and the default of *main()* is used when there is none. argv defaults to what *main()* parses, SAGE_DOCBUILD_OPTS included.

A directory named explicitly is not second-guessed: the lists have to describe the tree that was asked for, or none at all.

An option left without its value is reported rather than passed over, as the parser proper would report it.

Unique long-option abbreviations accepted by argparse designate the same source here.

The end-of-options marker leaves source-looking positional arguments to argparse instead of treating them as options.

1.2 Documentation builders

Note

If you are a developer and want to build the SageMath documentation from source, refer to developer's guide.

This module is the starting point for building documentation, and is responsible to figure out what to build and with which options. The actual documentation build for each individual document is then done in a subprocess call to Sphinx, see *builder_helper()*. Note that

- The builders are configured with build_options.py;

- The Sphinx subprocesses are configured in `conf.py`.

DocBuilder is the base class of all Builders. It has builder helpers *html()*, *latex()*, *pdf()*, *inventory()*, etc, which are invoked depending on the output type. Each type corresponds with the Sphinx builder format, except that *pdf()* is Sphinx latex builder plus compiling latex to pdf. Note that Sphinx inventory builder is not native to Sphinx but provided by Sage. See *sage_docbuild.ext.inventory_builder*. The Sphinx inventory builder is a dummy builder with no actual output but produces doctree files in `$SAGE_DOC/doctrees` and `objects.inv` inventory files in `$SAGE_DOC/inventory`.

The reference manual is built in two passes, first by *ReferenceBuilder* with `inventory` output type and secondly with `html` output type. The *ReferenceBuilder* itself uses *ReferenceTopBuilder* and *ReferenceSubBuilder* to build subcomponents of the reference manual. The *ReferenceSubBuilder* examines the modules included in the subcomponent by comparing the modification times of the module files with the times saved in `local/share/doctree/reference.pickle` from the previous build. Then new rst files are generated for new and updated modules. See *get_new_and_updated_modules()*.

After Issue #31948, when Sage is built, *ReferenceBuilder* is not used and its responsibility is now taken by the `Makefile` in `$SAGE_ROOT/src/doc`.

class sage_docbuild.builders.**DocBuilder**(*name: str*, *options:* BuildOptions)

Bases: `object`

INPUT:

- `name` – the name of a document directory below `doc`, such as 'en/tutorial' or 'fr/tutorial'

changes(**args*, ***kwds*)

Build the documentation for output type `changes`.

clean(**args*)

documents_single_file = False

Whether this builder documents one file of its own rather than a manual; see *SingleFileBuilder* and *builder_helper()*.

html(**args*, ***kwds*)

Build the documentation for output type `html`.

htmlhelp(**args*, ***kwds*)

Build the documentation for output type `htmlhelp`.

inventory(**args*, ***kwds*)

Build the documentation for output type `inventory`.

json(**args*, ***kwds*)

Build the documentation for output type `json`.

latex(**args*, ***kwds*)

Build the documentation for output type `latex`.

linkcheck(**args*, ***kwds*)

Build the documentation for output type `linkcheck`.

pdf()

Build the PDF files for this document.

This is done by first (re)-building the LaTeX output, going into that LaTeX directory, and running 'make all-pdf' there.

EXAMPLES:

```
sage: from sage_docbuild.builders import DocBuilder
sage: from sage_docbuild.build_options import BuildOptions
sage: options = BuildOptions(source_dir = Path('src/doc'))
sage: builder = DocBuilder('tutorial', options)
sage: builder.pdf() #not tested
```

```
>>> from sage.all import *
>>> from sage_docbuild.builders import DocBuilder
>>> from sage_docbuild.build_options import BuildOptions
>>> options = BuildOptions(source_dir = Path('src/doc'))
>>> builder = DocBuilder('tutorial', options)
>>> builder.pdf() #not tested
```

pickle(**args*, ***kwds*)

Build the documentation for output type `pickle`.

web(**args*, ***kwds*)

alias of *pickle()*.

class sage_docbuild.builders.**ReferenceBuilder**(*name: str*, *options:* BuildOptions)

Bases: `object`

This class builds the reference manual. It uses DocBuilder to build the top-level page and ReferenceSubBuilder for each sub-component.

print_unincluded_modules()

Print the modules of the Sage library that the reference manual does not include.

A sub-document knows the modules of its own toctrees only, so running the command of *ReferenceSubBuilder* over each of them in turn would report every module that any *other* sub-document documents. The answer is the complement of their union.

class sage_docbuild.builders.**ReferenceSubBuilder**(*name: str*, *options:* BuildOptions)

Bases: *DocBuilder*

This class builds sub-components of the reference manual. It is responsible for making sure that the auto generated reST files for the Sage library are up to date.

When building any output, we must first go through and check to see if we need to update any of the autogenerated reST files. There are two cases where this would happen:

1. A new module gets added to one of the toctrees.
2. The actual module gets updated and possibly contains a new title.

auto_rest_filename(*module_name*)

Return the name of the file associated to a given module

EXAMPLES:

```
sage: from sage_docbuild.builders import ReferenceSubBuilder
sage: from sage_docbuild.build_options import BuildOptions
sage: options = BuildOptions(source_dir = Path('src/doc'))
sage: ReferenceSubBuilder("reference", options).auto_rest_filename("sage.
↪combinat.partition")
...Path('src/doc/en/reference/sage/combinat/partition.rst')
```

```
>>> from sage.all import *
>>> from sage_docbuild.builders import ReferenceSubBuilder
>>> from sage_docbuild.build_options import BuildOptions
>>> options = BuildOptions(source_dir = Path('src/doc'))
>>> ReferenceSubBuilder("reference", options).auto_rest_filename("sage.
↪combinat.partition")
...Path('src/doc/en/reference/sage/combinat/partition.rst')
```

cache_file()

Return the filename where the pickle of the reference cache is stored.

clean_auto()

Remove all autogenerated reST files.

get_all_included_modules()

Return an iterator for all modules which are included in the reference manual.

get_all_rst_files()

Return an iterator for all rst files which are not autogenerated.

get_cache()

Retrieve the reference cache which contains the options previously used by the reference builder.

If it doesn't exist, then we just return an empty dictionary. If it is corrupted, return an empty dictionary.

get_modified_modules()

Return an iterator for all the modules that have been modified since the documentation was last built.

get_module_docstring_title(*module_name*)

Return the title of the module from its docstring.

get_modules(*file*)

Given a reST file, return an iterator for all of the autogenerated reST files that it includes.

get_new_and_updated_modules()

Return an iterator for all new and updated modules that appear in the toctrees, and remove obsolete old modules.

get_sphinx_environment()

Return the Sphinx environment for this project.

get_unincluded_modules()

Return an iterator for all the modules in the Sage library which are not included in this document.

Beware that a module the reference manual documents elsewhere is not included *here*; *ReferenceBuilder.print_unincluded_modules()* answers for the manual as a whole.

print_included_modules()

Print all of the modules that are included in the Sage reference manual.

print_modified_modules()

Print a list of all the modules that have been modified since the documentation was last built.

print_new_and_updated_modules()

Print all the modules that appear in the toctrees that are newly included or updated.

print_unincluded_modules()

Print all of the modules which are not included in the Sage reference manual.

save_cache()

Pickle the current reference cache for later retrieval.

update_mtimes()

Update the modification times for reST files in the Sphinx environment for this project.

write_auto_rest_file(*module_name*)

Write the autogenerated reST file for module_name.

class sage_docbuild.builders.**ReferenceTopBuilder**(*name: str*, *options:* BuildOptions)

Bases: *DocBuilder*

This class builds the top-level page of the reference manual.

html()

Build the top-level document.

class sage_docbuild.builders.**SingleFileBuilder**(*path: str*, *options:* BuildOptions)

Bases: *DocBuilder*

This is the class used to build the documentation for a single user-specified file. If the file is called 'foo.py', then the documentation is built in `DIR/foo/` if the user passes the command line option "-o DIR", or in `DOT_SAGE/docbuild/foo/` otherwise.

documents_single_file = True

Whether this builder documents one file of its own rather than a manual; see *SingleFileBuilder* and *builder_helper()*.

pdf()

Build a PDF, recording success only once the whole build ends.

class sage_docbuild.builders.**WebsiteBuilder**(*name: str*, *options:* BuildOptions)

Bases: *DocBuilder*

clean()

When we clean the output for the website index, we need to remove all of the HTML that were placed in the parent directory.

In addition, remove the index file installed into the root doc directory.

html()

After we have finished building the website index page, we copy everything one directory up, that is, to the base diectory `html/en`.

In addition, an index file is installed into the root doc directory.

Thus we have three index.html files:

html/en/website/index.html (not used) html/en/index.html (base directory) index.html (root doc directory)

pdf()

Build the website hosting pdf docs.

sage_docbuild.builders.**build_many**(*target*, *args*, *processes=None*)

Thin wrapper around *sage_docbuild.utils.build_many()* which uses the docbuild settings `NUM_THREADS` and `ABORT_ON_ERROR`.

sage_docbuild.builders.**build_ref_doc**(*args*)

sage_docbuild.builders.**builder_helper**(*type*)

Return a function which builds the documentation for output type `type`.

sage_docbuild.builders.**get_all_documents**(*source*)

Return a list of all of the documents, relative to the source directory.

A document is a directory within one of the language subdirectories of `doc`.

EXAMPLES:

```
sage: from sage_docbuild.builders import get_all_documents
sage: from sage.env import SAGE_DOC_SRC
sage: documents = get_all_documents(Path(SAGE_DOC_SRC))
sage: Path('en/tutorial') in documents
True
```

```
>>> from sage.all import *
>>> from sage_docbuild.builders import get_all_documents
>>> from sage.env import SAGE_DOC_SRC
>>> documents = get_all_documents(Path(SAGE_DOC_SRC))
>>> Path('en/tutorial') in documents
True
```

A directory without the root source that Sphinx would build is not a document:

```
sage: import tempfile
sage: with tempfile.TemporaryDirectory() as directory:
....:     source = Path(directory)
....:     empty = source / 'en' / 'empty'
....:     complete = source / 'en' / 'complete'
....:     empty.mkdir(parents=True)
....:     complete.mkdir()
....:     _ = (complete / 'index.rst').write_text('Complete\n========\n')
....:     get_all_documents(source) == [Path('en/complete')]
True
```

```
>>> from sage.all import *
>>> import tempfile
>>> with tempfile.TemporaryDirectory() as directory:
...     source = Path(directory)
...     empty = source / 'en' / 'empty'
...     complete = source / 'en' / 'complete'
...     empty.mkdir(parents=True)
...     complete.mkdir()
...     _ = (complete / 'index.rst').write_text('Complete\n========\n')
...     get_all_documents(source) == [Path('en/complete')]
True
```

sage_docbuild.builders.**get_all_reference_documents**(*source*)

Return a list of all reference manual documents to build, relative to the specified source directory.

We add a document if it's a subdirectory of the manual's directory and contains a file named 'index.rst'.

The order corresponds to the order in which the documents should be built.

EXAMPLES:

```
sage: from sage_docbuild.builders import get_all_reference_documents
sage: from sage.env import SAGE_DOC_SRC
sage: documents = get_all_reference_documents(Path(SAGE_DOC_SRC) / 'en')
sage: Path('reference/algebras') in documents
True
```

```
>>> from sage.all import *
>>> from sage_docbuild.builders import get_all_reference_documents
>>> from sage.env import SAGE_DOC_SRC
>>> documents = get_all_reference_documents(Path(SAGE_DOC_SRC) / 'en')
>>> Path('reference/algebras') in documents
True
```

sage_docbuild.builders.**get_builder**(*name*, *options*)

Return an appropriate *Builder* object for the document `name`.

DocBuilder and its subclasses do all the real work in building the documentation.

The name is one of those that *get_documents()* lists, so an English document may be named without its `en/` prefix.

EXAMPLES:

```
sage: from sage.env import SAGE_DOC_SRC
sage: from sage_docbuild.build_options import BuildOptions
sage: from sage_docbuild.builders import get_builder
sage: options = BuildOptions(source_dir=Path(SAGE_DOC_SRC))
sage: get_builder('developer', options).name
'en/developer'
sage: get_builder('en/developer', options).name
'en/developer'
sage: get_builder('fr/tutorial', options).name
'fr/tutorial'
```

```
>>> from sage.all import *
>>> from sage.env import SAGE_DOC_SRC
>>> from sage_docbuild.build_options import BuildOptions
>>> from sage_docbuild.builders import get_builder
>>> options = BuildOptions(source_dir=Path(SAGE_DOC_SRC))
>>> get_builder('developer', options).name
'en/developer'
>>> get_builder('en/developer', options).name
'en/developer'
>>> get_builder('fr/tutorial', options).name
'fr/tutorial'
```

The website has a builder of its own, and reaches it under either name:

```
sage: type(get_builder('website', options)).__name__
'WebsiteBuilder'
sage: get_builder('website', options).name
'en/website'
```

```
>>> from sage.all import *
>>> type(get_builder('website', options)).__name__
'WebsiteBuilder'
>>> get_builder('website', options).name
'en/website'
```

A document merely ending in `reference` is not mistaken for the Sage reference manual:

```
sage: import contextlib, io, tempfile
sage: with tempfile.TemporaryDirectory() as directory:
....:     source = Path(directory)
....:     document = source / 'fr' / 'myreference'
....:     document.mkdir(parents=True)
....:     _ = (document / 'index.rst').write_text('Title\n=====\n')
....:     english = source / 'en'
....:     english.mkdir()
....:     _ = (english / 'index.rst').write_text('English\n=======\n')
....:     local = BuildOptions(source_dir=source)
....:     answer = get_builder('fr/myreference', local)
....:     with contextlib.redirect_stdout(io.StringIO()):
....:         try:
....:             get_builder('reference/..', local)
....:         except SystemExit:
....:             traversal_rejected = True
sage: type(answer).__name__, answer.name, traversal_rejected
('DocBuilder', 'fr/myreference', True)
```

```
>>> from sage.all import *
>>> import contextlib, io, tempfile
>>> with tempfile.TemporaryDirectory() as directory:
...     source = Path(directory)
...     document = source / 'fr' / 'myreference'
...     document.mkdir(parents=True)
...     _ = (document / 'index.rst').write_text('Title\n=====\n')
...     english = source / 'en'
...     english.mkdir()
...     _ = (english / 'index.rst').write_text('English\n=======\n')
...     local = BuildOptions(source_dir=source)
...     answer = get_builder('fr/myreference', local)
...     with contextlib.redirect_stdout(io.StringIO()):
...         try:
...             get_builder('reference/..', local)
...         except SystemExit:
...             traversal_rejected = True
>>> type(answer).__name__, answer.name, traversal_rejected
('DocBuilder', 'fr/myreference', True)
```

sage_docbuild.builders.**get_documents**(*source*)

Return the documents that the documentation builder accepts as command-line arguments, in the order in which they are built.

These are the documents of *get_all_documents()*, named the way the command line names them: the English ones without their language prefix, and the reference manual - which *get_all_documents()* leaves out, its top level having a builder of its own - in front.

EXAMPLES:

```
sage: from sage.env import SAGE_DOC_SRC
sage: from sage_docbuild.builders import get_documents
sage: documents = get_documents(Path(SAGE_DOC_SRC))
sage: documents[0]
'reference'
sage: 'tutorial' in documents
True
sage: 'en/tutorial' in documents
False
```

```
>>> from sage.all import *
>>> from sage.env import SAGE_DOC_SRC
>>> from sage_docbuild.builders import get_documents
>>> documents = get_documents(Path(SAGE_DOC_SRC))
>>> documents[Integer(0)]
'reference'
>>> 'tutorial' in documents
True
>>> 'en/tutorial' in documents
False
```

sage_docbuild.builders.**load_single_file**(**args*, ***kwds*)

Import the file `path` under the module name `name`, and return it.

This is what the configuration of a single-file build runs when importing `name` would read another file - the same module of a second checkout, say - or no file at all. The modules beside `path` come before the ones of that other copy, so that what the file imports is what sits next to it; a module of that copy which is loaded already is dropped, since an import would answer with it and never read this directory.

Modules that `path` reaches by any other route are the ones already loaded, which is what the caller warns about.

For an ordinary module, the already-active containing package is retained; its initializer is not re-executed from the alternate tree. When `path` is itself `__init__.py`, that package initializer is the explicit target and is replaced together with its children.

EXAMPLES:

```
sage: import os, sys, tempfile
sage: from sage_docbuild.builders import (load_single_file,
....:     _clear_single_file_imports)
sage: with tempfile.TemporaryDirectory() as directory:
....:     package = os.path.join(directory, 'pkg_2718')
....:     os.mkdir(package)
....:     _ = open(os.path.join(package, '__init__.py'), 'w').close()
....:     with open(os.path.join(package, 'helper.py'), 'w') as f:
....:         _ = f.write('answer = 42\n')
....:     with open(os.path.join(package, 'main.py'), 'w') as f:
....:         _ = f.write('import _imp\nfrom .helper import answer\n'
....:                     'lock_held = _imp.lock_held()\n')
....:     sys.path.insert(0, directory)
....:     module = load_single_file('pkg_2718.main',
....:                               os.path.join(package, 'main.py'))
....:     _clear_single_file_imports()
```

(continues on next page)

(continued from previous page)

```
....:     sys.path.remove(directory)
sage: module.answer
42
sage: module.__name__
'pkg_2718.main'
sage: module.lock_held
False
```

```
>>> from sage.all import *
>>> import os, sys, tempfile
>>> from sage_docbuild.builders import (load_single_file,
...     _clear_single_file_imports)
>>> with tempfile.TemporaryDirectory() as directory:
...     package = os.path.join(directory, 'pkg_2718')
...     os.mkdir(package)
...     _ = open(os.path.join(package, '__init__.py'), 'w').close()
...     with open(os.path.join(package, 'helper.py'), 'w') as f:
...         _ = f.write('answer = 42\n')
...     with open(os.path.join(package, 'main.py'), 'w') as f:
...         _ = f.write('import _imp\nfrom .helper import answer\n'
...                     'lock_held = _imp.lock_held()\n')
...     sys.path.insert(Integer(0), directory)
...     module = load_single_file('pkg_2718.main',
...                               os.path.join(package, 'main.py'))
...     _clear_single_file_imports()
...     sys.path.remove(directory)
>>> module.answer
42
>>> module.__name__
'pkg_2718.main'
>>> module.lock_held
False
```

Already-loaded children and descendants of another copy are replaced, and a failed replacement restores them:

```
sage: import importlib
sage: from sage_docbuild.builders import (_SiblingFinder,
....:     _module_path)
sage: with tempfile.TemporaryDirectory() as active, \
....:      tempfile.TemporaryDirectory() as alternate:
....:     active_pkg = Path(active) / 'pkg_shadow_2718'
....:     alternate_pkg = Path(alternate) / 'pkg_shadow_2718'
....:     (active_pkg / 'nested').mkdir(parents=True)
....:     (alternate_pkg / 'nested').mkdir(parents=True)
....:     for package in (active_pkg, alternate_pkg,
....:                     active_pkg / 'nested', alternate_pkg / 'nested'):
....:         _ = (package / '__init__.py').write_text('')
....:     _ = (active_pkg / 'helper.py').write_text("VALUE = 'active'\n")
....:     _ = (active_pkg / 'nested' / 'helper.py').write_text(
....:         "VALUE = 'active nested'\n")
....:     _ = (active_pkg / 'broken.py').write_text("VALUE = 'old'\n")
....:     _ = (active_pkg / 'fallback.py').write_text(
```

(continues on next page)

(continued from previous page)

```
....:         "VALUE = 'active fallback'\n")
....:     _ = (Path(active) / 'unrelated_tx_2718.py').write_text(
....:         "VALUE = 'preserved'\n")
....:     _ = (alternate_pkg / 'helper.py').write_text(
....:         "VALUE = 'alternate'\n")
....:     _ = (alternate_pkg / 'nested' / 'helper.py').write_text(
....:         "VALUE = 'alternate nested'\n")
....:     _ = (alternate_pkg / 'main.py').write_text(
....:         'from . import helper\n'
....:         'from .nested.helper import VALUE as NESTED\n'
....:         'VALUE = helper.VALUE\n')
....:     sys.path.insert(0, active)
....:     package = importlib.import_module('pkg_shadow_2718')
....:     old_helper = importlib.import_module('pkg_shadow_2718.helper')
....:     old_nested = importlib.import_module(
....:         'pkg_shadow_2718.nested.helper')
....:     main = load_single_file(
....:         'pkg_shadow_2718.main', str(alternate_pkg / 'main.py'))
....:     values = (main.VALUE, main.NESTED,
....:               package.helper is not old_helper,
....:               sys.modules['pkg_shadow_2718.nested.helper'] is not old_
↪nested)
....:     _clear_single_file_imports()
....:     active_restored = (
....:         package.helper is old_helper
....:         and sys.modules['pkg_shadow_2718.nested.helper'] is old_nested
....:         and _module_path('pkg_shadow_2718.helper')
....:             == os.path.realpath(active_pkg / 'helper.py')
....:         and not any(isinstance(item, _SiblingFinder)
....:                     and item.package == 'pkg_shadow_2718'
....:                     for item in sys.meta_path))
....:     old_broken = importlib.import_module('pkg_shadow_2718.broken')
....:     _ = sys.modules.pop('unrelated_tx_2718', None)
....:     _ = (alternate_pkg / 'broken.py').write_text(
....:         "import unrelated_tx_2718\n"
....:         "from . import fallback\n"
....:         "VALUE = 'partial'\nraise RuntimeError('broken')\n")
....:     meta_path = list(sys.meta_path)
....:     try:
....:         load_single_file('pkg_shadow_2718.broken',
....:                          str(alternate_pkg / 'broken.py'))
....:     except RuntimeError:
....:         restored = (sys.modules['pkg_shadow_2718.broken'] is old_broken
....:                     and package.broken is old_broken
....:                     and sys.meta_path == meta_path)
....:     unrelated_preserved = 'unrelated_tx_2718' in sys.modules
....:     fallback_preserved = (
....:         package.fallback
....:         is sys.modules['pkg_shadow_2718.fallback'])
....:     _ = sys.modules.pop('unrelated_tx_2718', None)
....:     sys.path.remove(active)
....:     for loaded in list(sys.modules):
```

(continues on next page)

(continued from previous page)

```
....:         if loaded == 'pkg_shadow_2718' or loaded.startswith(
....:                 'pkg_shadow_2718.'):
....:             del sys.modules[loaded]
....:     sys.meta_path[:] = [finder for finder in sys.meta_path
....:                         if not (isinstance(finder, _SiblingFinder)
....:                                 and finder.package == 'pkg_shadow_2718')]
sage: (values, active_restored, restored, unrelated_preserved,
....:  fallback_preserved)
(('alternate', 'alternate nested', True, True), True, True, True, True)
```

```
>>> from sage.all import *
>>> import importlib
>>> from sage_docbuild.builders import (_SiblingFinder,
...     _module_path)
>>> with tempfile.TemporaryDirectory() as active,      tempfile.
↪TemporaryDirectory() as alternate:
...     active_pkg = Path(active) / 'pkg_shadow_2718'
...     alternate_pkg = Path(alternate) / 'pkg_shadow_2718'
...     (active_pkg / 'nested').mkdir(parents=True)
...     (alternate_pkg / 'nested').mkdir(parents=True)
...     for package in (active_pkg, alternate_pkg,
...                     active_pkg / 'nested', alternate_pkg / 'nested'):
...         _ = (package / '__init__.py').write_text('')
...     _ = (active_pkg / 'helper.py').write_text("VALUE = 'active'\n")
...     _ = (active_pkg / 'nested' / 'helper.py').write_text(
...         "VALUE = 'active nested'\n")
...     _ = (active_pkg / 'broken.py').write_text("VALUE = 'old'\n")
...     _ = (active_pkg / 'fallback.py').write_text(
...         "VALUE = 'active fallback'\n")
...     _ = (Path(active) / 'unrelated_tx_2718.py').write_text(
...         "VALUE = 'preserved'\n")
...     _ = (alternate_pkg / 'helper.py').write_text(
...         "VALUE = 'alternate'\n")
...     _ = (alternate_pkg / 'nested' / 'helper.py').write_text(
...         "VALUE = 'alternate nested'\n")
...     _ = (alternate_pkg / 'main.py').write_text(
...         'from . import helper\n'
...         'from .nested.helper import VALUE as NESTED\n'
...         'VALUE = helper.VALUE\n')
...     sys.path.insert(Integer(0), active)
...     package = importlib.import_module('pkg_shadow_2718')
...     old_helper = importlib.import_module('pkg_shadow_2718.helper')
...     old_nested = importlib.import_module(
...         'pkg_shadow_2718.nested.helper')
...     main = load_single_file(
...         'pkg_shadow_2718.main', str(alternate_pkg / 'main.py'))
...     values = (main.VALUE, main.NESTED,
...               package.helper is not old_helper,
...               sys.modules['pkg_shadow_2718.nested.helper'] is not old_nested)
...     _clear_single_file_imports()
...     active_restored = (
...         package.helper is old_helper
```

(continues on next page)

(continued from previous page)

```
...         and sys.modules['pkg_shadow_2718.nested.helper'] is old_nested
...         and _module_path('pkg_shadow_2718.helper')
...             == os.path.realpath(active_pkg / 'helper.py')
...         and not any(isinstance(item, _SiblingFinder)
...                     and item.package == 'pkg_shadow_2718'
...                     for item in sys.meta_path))
...     old_broken = importlib.import_module('pkg_shadow_2718.broken')
...     _ = sys.modules.pop('unrelated_tx_2718', None)
...     _ = (alternate_pkg / 'broken.py').write_text(
...         "import unrelated_tx_2718\n"
...         "from . import fallback\n"
...         "VALUE = 'partial'\nraise RuntimeError('broken')\n")
...     meta_path = list(sys.meta_path)
...     try:
...         load_single_file('pkg_shadow_2718.broken',
...                          str(alternate_pkg / 'broken.py'))
...     except RuntimeError:
...         restored = (sys.modules['pkg_shadow_2718.broken'] is old_broken
...                     and package.broken is old_broken
...                     and sys.meta_path == meta_path)
...     unrelated_preserved = 'unrelated_tx_2718' in sys.modules
...     fallback_preserved = (
...         package.fallback
...         is sys.modules['pkg_shadow_2718.fallback'])
...     _ = sys.modules.pop('unrelated_tx_2718', None)
...     sys.path.remove(active)
...     for loaded in list(sys.modules):
...         if loaded == 'pkg_shadow_2718' or loaded.startswith(
...                 'pkg_shadow_2718.'):
...             del sys.modules[loaded]
...     sys.meta_path[:] = [finder for finder in sys.meta_path
...                         if not (isinstance(finder, _SiblingFinder)
...                                 and finder.package == 'pkg_shadow_2718')]
>>> (values, active_restored, restored, unrelated_preserved,
...  fallback_preserved)
(('alternate', 'alternate nested', True, True), True, True, True, True)
```

A package initializer replaces its loaded children as well:

```
sage: with tempfile.TemporaryDirectory() as active, \
....:      tempfile.TemporaryDirectory() as alternate:
....:     for root, value in ((active, 'active'),
....:                         (alternate, 'alternate')):
....:         package = Path(root) / 'pkg_init_2718'
....:         package.mkdir()
....:         _ = (package / '__init__.py').write_text(
....:             'from .helper import VALUE\n')
....:         _ = (package / 'helper.py').write_text(
....:             f"VALUE = {value!r}\n")
....:     sys.path.insert(0, active)
....:     old_package = importlib.import_module('pkg_init_2718')
....:     old_helper = importlib.import_module('pkg_init_2718.helper')
```

(continues on next page)

(continued from previous page)

```
....:     package = load_single_file(
....:         'pkg_init_2718',
....:         str(Path(alternate) / 'pkg_init_2718' / '__init__.py'))
....:     answer = (package.VALUE,
....:               sys.modules['pkg_init_2718.helper'] is not old_helper)
....:     _clear_single_file_imports()
....:     sys.path.remove(active)
....:     for loaded in list(sys.modules):
....:         if loaded == 'pkg_init_2718' or loaded.startswith(
....:                 'pkg_init_2718.'):
....:             del sys.modules[loaded]
....:     sys.meta_path[:] = [finder for finder in sys.meta_path
....:                         if not (isinstance(finder, _SiblingFinder)
....:                                 and finder.package == 'pkg_init_2718')]
sage: answer
('alternate', True)
```

```
>>> from sage.all import *
>>> with tempfile.TemporaryDirectory() as active,      tempfile.
↪TemporaryDirectory() as alternate:
...     for root, value in ((active, 'active'),
...                         (alternate, 'alternate')):
...         package = Path(root) / 'pkg_init_2718'
...         package.mkdir()
...         _ = (package / '__init__.py').write_text(
...             'from .helper import VALUE\n')
...         _ = (package / 'helper.py').write_text(
...             f"VALUE = {value!r}\n")
...     sys.path.insert(Integer(0), active)
...     old_package = importlib.import_module('pkg_init_2718')
...     old_helper = importlib.import_module('pkg_init_2718.helper')
...     package = load_single_file(
...         'pkg_init_2718',
...         str(Path(alternate) / 'pkg_init_2718' / '__init__.py'))
...     answer = (package.VALUE,
...               sys.modules['pkg_init_2718.helper'] is not old_helper)
...     _clear_single_file_imports()
...     sys.path.remove(active)
...     for loaded in list(sys.modules):
...         if loaded == 'pkg_init_2718' or loaded.startswith(
...                 'pkg_init_2718.'):
...             del sys.modules[loaded]
...     sys.meta_path[:] = [finder for finder in sys.meta_path
...                         if not (isinstance(finder, _SiblingFinder)
...                                 and finder.package == 'pkg_init_2718')]
>>> answer
('alternate', True)
```

If importing a previously absent containing package is part of a failed load, that package and its canonical child binding are rolled back too:

```
sage: with tempfile.TemporaryDirectory() as directory:
```

(continues on next page)

(continued from previous page)

```
....:     package = Path(directory) / 'pkg_parent_2718'
....:     package.mkdir()
....:     _ = (package / '__init__.py').write_text(
....:         'from . import side\n')
....:     _ = (package / 'side.py').write_text('VALUE = 42\n')
....:     broken = package / 'broken.py'
....:     _ = broken.write_text("raise RuntimeError('broken')\n")
....:     sys.path.insert(0, directory)
....:     try:
....:         load_single_file('pkg_parent_2718.broken', str(broken))
....:     except RuntimeError:
....:         rolled_back = not any(
....:             loaded == 'pkg_parent_2718'
....:             or loaded.startswith('pkg_parent_2718.')
....:             for loaded in sys.modules)
....:     sys.path.remove(directory)
sage: rolled_back
True
```

```
>>> from sage.all import *
>>> with tempfile.TemporaryDirectory() as directory:
...     package = Path(directory) / 'pkg_parent_2718'
...     package.mkdir()
...     _ = (package / '__init__.py').write_text(
...         'from . import side\n')
...     _ = (package / 'side.py').write_text('VALUE = 42\n')
...     broken = package / 'broken.py'
...     _ = broken.write_text("raise RuntimeError('broken')\n")
...     sys.path.insert(Integer(0), directory)
...     try:
...         load_single_file('pkg_parent_2718.broken', str(broken))
...     except RuntimeError:
...         rolled_back = not any(
...             loaded == 'pkg_parent_2718'
...             or loaded.startswith('pkg_parent_2718.')
...             for loaded in sys.modules)
...     sys.path.remove(directory)
>>> rolled_back
True
```

1.3 Build options

This module defines options for building Sage documentation.

class sage_docbuild.build_options.**BuildOptions**(***kwargs*)

Bases: Namespace

output_dir: **Path**

output_dir_given: **bool**

Whether `output_dir` was given on the command line, as opposed to being filled in with the default. Single-file builds use their own default, so that they never write into the installed documentation.

source_dir: Path

1.4 Sphinx build script

This is Sage's version of the `sphinx-build` script. We redirect `stdout` and `stderr` to our own logger, and remove some unwanted chatter.

class sage_docbuild.sphinxbuild.**KnownCitation**(*labels*)

Bases: `object`

Match the warning about a citation that Sage defines.

A single file is documented without the multidocs extension, hence without the bibliography of the reference manual, so a citation of it - and most Sage files cite something - has nothing to resolve against; see #29651. Ignoring every such warning would hide the citations that are misspelled, which are the ones worth reporting, so the labels that the sources define are consulted instead; see *citation_labels()*.

Instances answer to `search()`, as the compiled patterns they are used among do.

EXAMPLES:

```
sage: from sage_docbuild.sphinxbuild import KnownCitation
sage: known = KnownCitation({'Cohen1996'})
sage: bool(known.search('WARNING: citation not found: Cohen1996'))
True
sage: bool(known.search('WARNING: citation not found: Cohen1997'))
False
sage: bool(known.search('WARNING: citation not found: cohen1996'))
True
```

```
>>> from sage.all import *
>>> from sage_docbuild.sphinxbuild import KnownCitation
>>> known = KnownCitation({'Cohen1996'})
>>> bool(known.search('WARNING: citation not found: Cohen1996'))
True
>>> bool(known.search('WARNING: citation not found: Cohen1997'))
False
>>> bool(known.search('WARNING: citation not found: cohen1996'))
True
```

Sphinx colors what it writes to a terminal, and the label is read past the escape sequences:

```
sage: bool(known.search(
....:     '\x1b[91mWARNING: citation not found: Cohen1996\x1b[39;49;00m'))
True
```

```
>>> from sage.all import *
>>> bool(known.search(
...     '\x1b[91mWARNING: citation not found: Cohen1996\x1b[39;49;00m'))
True
```

search(*line*)

class sage_docbuild.sphinxbuild.**SageSphinxLogger**(*stream*, *prefix*, * *(Keyword-only parameters separator (PEP 3102))*, *warnings_are_errors=False*, *ignored_warnings=()*)

Bases: object

This implements the file object interface to serve as sys.stdout/sys.stderr replacement.

ansi_escape_sequence = re.compile('\n \\x1b # ESC\n \\[# CSI sequence starts\n [0-?]* # parameter bytes\n [-/]* # intermediate bytes\n [@-~] # final byte\n ', re.VERBOSE)

ansi_escape_sequence_color = re.compile('\n \\x1b # ESC\n \\[# CSI sequence starts\n [0-9;]* # parameter bytes\n # intermediate bytes\n m # final byte\n ', re.VERBOSE)

close()

closed = False

encoding = None

flush()

isatty()

mode = 'w'

name = '<log>'

newlines = None

prefix_len = 9

raise_errors()

Raise an exceptions if any errors have been found while parsing the Sphinx output.

EXAMPLES:

```
sage: from sys import stdout
sage: from sage_docbuild.sphinxbuild import SageSphinxLogger
sage: logger = SageSphinxLogger(stdout, "doctesting")
sage: logger._log_line("This is a SEVERE error\n")
[doctestin] This is a SEVERE error
sage: logger.raise_errors()
Traceback (most recent call last):
...
OSError: This is a SEVERE error
```

```
>>> from sage.all import *
>>> from sys import stdout
>>> from sage_docbuild.sphinxbuild import SageSphinxLogger
>>> logger = SageSphinxLogger(stdout, "doctesting")
>>> logger._log_line("This is a SEVERE error\n")
[doctestin] This is a SEVERE error
>>> logger.raise_errors()
Traceback (most recent call last):
...
OSError: This is a SEVERE error
```

softspace = 0

write(*str*)

writelines(*sequence*)

sage_docbuild.sphinxbuild.**citation_labels**(*single_file_path=None*, *single_file_source_root=None*)

Return every citation label that the sources of Sage define, or `None` when the bibliography of the reference manual is not among the sources.

Most citations are collected in a reference manual, and the rest are written in the docstring defining them. Both are parsed as reStructuredText so that examples and comments are not mistaken for definitions. For a file of a second checkout, that checkout is searched as well as the active one.

No answer is cached: a docbuild process can build more than one file, edit a source between builds, or switch the source root in its environment.

EXAMPLES:

```
sage: from sage_docbuild.sphinxbuild import citation_labels
sage: labels = citation_labels()
sage: labels is None or 'AB2007' in labels
True
```

```
>>> from sage.all import *
>>> from sage_docbuild.sphinxbuild import citation_labels
>>> labels = citation_labels()
>>> labels is None or 'AB2007' in labels
True
```

sage_docbuild.sphinxbuild.**runsphinx**(*argv*, *prefix*, *warnings_are_errors*, *first_pass*, *is_inventory*, *single_file*, *single_file_path*, *single_file_source_root*)

Run `sphinx-build` with the arguments `argv`, logging its output.

INPUT:

- `argv` – list of arguments for `sphinx-build`, without the program name; the last one is the output directory
- `prefix` – string used to tag the output lines; defaults to the name of the output directory
- `warnings_are_errors` – whether a warning makes the build fail. The LaTeX builder warns about markup that it cannot translate, which is not a reason to stop.
- `first_pass` – whether this build runs before the inventories of the other documents have been written
- `is_inventory` – whether this build only writes an inventory
- `single_file` – whether this build documents one file of its own, outside of any manual
- `single_file_path` – the file being documented, when `single_file` is true
- `single_file_source_root` – the import root of `single_file_path`

A failed build raises `OSError`, either because Sphinx reported a nonzero exit status, or because it logged a diagnostic that Sage treats as an error.

sage_docbuild.sphinxbuild.**single_file_ignored_warnings**(*single_file_path=None*, *single_file_source_root=None*)

Return the warning patterns that a file documented on its own is built with.

Its citations are the ones that *KnownCitation* accounts for. Where the sources carry no bibliography to tell one citation from another, all of them are ignored.

EXAMPLES:

```
sage: from sage_docbuild.sphinxbuild import single_file_ignored_warnings
sage: all(hasattr(pattern, 'search')
....:     for pattern in single_file_ignored_warnings())
True
```

```
>>> from sage.all import *
>>> from sage_docbuild.sphinxbuild import single_file_ignored_warnings
>>> all(hasattr(pattern, 'search')
...     for pattern in single_file_ignored_warnings())
True
```

sage_docbuild.sphinxbuild.**term_width_line**(*text*)

1.5 Sphinx build configuration

This file contains configuration needed to customize Sphinx input and output behavior.

sage_docbuild.conf.**add_page_context**(*app*, *pagename*, *templatename*, *context*, *doctree*)

sage_docbuild.conf.**linkcode_resolve**(*domain*, *info*)

sage_docbuild.conf.**reference_subdocument**(*directory=None*)

Return the configuration values proper to one sub-document of the reference manual.

Every sub-document is a Sphinx project of its own, with a configuration file that takes the shared configuration from this module and then the values returned here, which only depend on where the sub-document lives:

```
from sage_docbuild.conf import *
from sage_docbuild.conf import reference_subdocument

globals().update(reference_subdocument())
```

Anything proper to a single sub-document belongs in its configuration file, after that call.

INPUT:

- `directory` – the directory of the sub-document; defaults to the current directory, which is the one holding the configuration file that Sphinx is reading

As a side effect, the shared `html_theme_options` and `latex_elements` are given the entries that depend on the sub-document; a Sphinx run builds a single document, so they cannot be shared.

EXAMPLES:

```
sage: import os, tempfile
sage: from sage_docbuild.conf import reference_subdocument
sage: def subdocument(name, index):
....:     directory = os.path.join(tempfile.mkdtemp(), name)
....:     os.mkdir(directory)
....:     with open(os.path.join(directory, 'index.rst'), 'w') as f:
....:         _ = f.write(index)
```

(continues on next page)

(continued from previous page)

```
....:     return reference_subdocument(directory)

sage: config = subdocument('algebras', 'Algebras\n========\n')
sage: config['htmlhelp_basename']
'algebras'
sage: config['html_title']
'Algebras'
sage: config['latex_documents']
[('index', 'algebras.tex', 'Algebras', 'The Sage Development Team', 'manual')]
sage: config['multidocs_is_master']
False
```

```
>>> from sage.all import *
>>> import os, tempfile
>>> from sage_docbuild.conf import reference_subdocument
>>> def subdocument(name, index):
...     directory = os.path.join(tempfile.mkdtemp(), name)
...     os.mkdir(directory)
...     with open(os.path.join(directory, 'index.rst'), 'w') as f:
...         _ = f.write(index)
...     return reference_subdocument(directory)

>>> config = subdocument('algebras', 'Algebras\n========\n')
>>> config['htmlhelp_basename']
'algebras'
>>> config['html_title']
'Algebras'
>>> config['latex_documents']
[('index', 'algebras.tex', 'Algebras', 'The Sage Development Team', 'manual')]
>>> config['multidocs_is_master']
False
```

A title in backticks is math, which the HTML title writes with dollars:

```
sage: subdocument('padics', '`p`-adics\n=========\n')['html_title']
'$p$-adics'
```

```
>>> from sage.all import *
>>> subdocument('padics', '`p`-adics\n=========\n')['html_title']
'$p$-adics'
```

Without a title, the name of the directory is used:

```
sage: subdocument('padics', 'No title here.\n')['html_title']
'Padics'
```

```
>>> from sage.all import *
>>> subdocument('padics', 'No title here.\n')['html_title']
'Padics'
```

sage_docbuild.conf.**setup**(*app*)

1.6 Utilities

exception sage_docbuild.utils.**RemoteException**(*tb: str*)

Bases: Exception

Raised if an exception occurred in one of the child processes.

tb: str

class sage_docbuild.utils.**RemoteExceptionWrapper**(*exc: BaseException*)

Bases: object

Used by child processes to capture exceptions thrown during execution and report them to the main process, including the correct traceback.

exc: BaseException

tb: str

exception sage_docbuild.utils.**WorkerDiedException**(*message: str | None*, *original_exception: BaseException | None = None*)

Bases: RuntimeError

Raised if a worker process dies unexpected.

original_exception: BaseException | None

sage_docbuild.utils.**build_many**(*target*, *args*, *processes=None*)

Map a list of arguments in `args` to a single-argument target function `target` in parallel using `multiprocessing.cpu_count()` (or `processes` if given) simultaneous processes.

This is a simplified version of `multiprocessing.Pool.map` from the Python standard library which avoids a couple of its pitfalls. In particular, it can abort (with a `RuntimeError`) without hanging if one of the worker processes unexpectedly dies. It also has semantics equivalent to `maxtasksperchild=1`; that is, one process is started per argument. As such, this is inefficient for processing large numbers of fast tasks, but appropriate for running longer tasks (such as doc builds) which may also require significant cleanup.

It also avoids starting new processes from a pthread, which results in at least one known issue:

- When PARI is built with multi-threading support, forking a Sage process from a thread leaves the main Pari interface instance broken (see Issue #26608#comment:38).

In the future this may be replaced by a generalized version of the more robust parallel processing implementation from `sage.doctest.forker`.

EXAMPLES:

```
sage: from sage_docbuild.utils import build_many
sage: def target(N):
....:     import time
....:     time.sleep(float(0.1))
....:     print('Processed task %s' % N)
sage: _ = build_many(target, range(8), processes=8)
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
```

(continues on next page)

(continued from previous page)

```
Processed task ...
Processed task ...
```

```
>>> from sage.all import *
>>> from sage_docbuild.utils import build_many
>>> def target(N):
...     import time
...     time.sleep(float(RealNumber('0.1')))
...     print('Processed task %s' % N)
>>> _ = build_many(target, range(Integer(8)), processes=Integer(8))
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
Processed task ...
```

This version can also return a result, and thus can be used as a replacement for `multiprocessing.Pool.map` (i.e. it still blocks until the result is ready):

```
sage: def square(N):
....:     return N * N
sage: build_many(square, range(100))
[0, 1, 4, 9, ..., 9604, 9801]
```

```
>>> from sage.all import *
>>> def square(N):
...     return N * N
>>> build_many(square, range(Integer(100)))
[0, 1, 4, 9, ..., 9604, 9801]
```

If the target function raises an exception in any of the workers, `build_many` raises that exception and all other results are discarded. Any in-progress tasks may still be allowed to complete gracefully before the exception is raised:

```
sage: def target(N):
....:     import time, os, signal
....:     if N == 4:
....:         # Task 4 is a poison pill
....:         1 / 0
....:     else:
....:         time.sleep(float(0.5))
....:         print('Processed task %s' % N)
```

```
>>> from sage.all import *
>>> def target(N):
...     import time, os, signal
...     if N == Integer(4):
...         # Task 4 is a poison pill
...         Integer(1) / Integer(0)
```

(continues on next page)

(continued from previous page)

```
...     else:
...         time.sleep(float(RealNumber('0.5')))
...         print('Processed task %s' % N)
```

Note: In practice this test might still show output from the other worker processes before the poison-pill is executed. It may also display the traceback from the failing process on stderr. However, due to how the doctest runner works, the doctest will only expect the final exception:

```
sage: build_many(target, range(8), processes=8)
Traceback (most recent call last):
...
    raise ZeroDivisionError("rational division by zero")
 ZeroDivisionError: rational division by zero
...
    raise worker_exc.original_exception
ZeroDivisionError: rational division by zero
```

```
>>> from sage.all import *
>>> build_many(target, range(Integer(8)), processes=Integer(8))
Traceback (most recent call last):
...
    raise ZeroDivisionError("rational division by zero")
 ZeroDivisionError: rational division by zero
...
    raise worker_exc.original_exception
ZeroDivisionError: rational division by zero
```

Similarly, if one of the worker processes dies unexpectedly otherwise exits non-zero (e.g. killed by a signal) any in-progress tasks will be completed gracefully, but then a `RuntimeError` is raised and pending tasks are not started:

```
sage: def target(N):
....:     import time, os, signal
....:     if N == 4:
....:         # Task 4 is a poison pill
....:         os.kill(os.getpid(), signal.SIGKILL)
....:     else:
....:         time.sleep(float(0.5))
....:         print('Processed task %s' % N)
sage: build_many(target, range(8), processes=8)
Traceback (most recent call last):
...
WorkerDiedException: worker for 4 died with non-zero exit code -9
```

```
>>> from sage.all import *
>>> def target(N):
...     import time, os, signal
...     if N == Integer(4):
...         # Task 4 is a poison pill
...         os.kill(os.getpid(), signal.SIGKILL)
...     else:
...         time.sleep(float(RealNumber('0.5')))
```

(continues on next page)

(continued from previous page)

```
...         print('Processed task %s' % N)
>>> build_many(target, range(Integer(8)), processes=Integer(8))
Traceback (most recent call last):
...
WorkerDiedException: worker for 4 died with non-zero exit code -9
```

CHAPTER

TWO

SPHINX EXTENSIONS

2.1 Inventory builder

A customized builder which only generates intersphinx "object.inv" inventory files. The documentation files are not written.

class sage_docbuild.ext.inventory_builder.**InventoryBuilder**(*app: sphinx.application.Sphinx*, *env: sphinx.environment.BuildEnvironment*)

Bases: DummyBuilder

A customized builder which only generates intersphinx "object.inv" inventory files. The documentation files are not written.

epilog: ClassVar[str] = 'The inventory file is in %(outdir)s.'

The message emitted upon successful build completion. This can be a printf-style template string with the following keys: outdir, project

finish()

Only write the inventory files.

format: ClassVar[str] = 'inventory'

The output-format identifier used by the inventory builder.

get_outdated_docs()

Return an iterable of output files that are outdated.

get_target_uri(*docname*, *typ=None*)

Return the target URI for a document name.

name: ClassVar[str] = 'inventory'

The builder's name. This is the value used to select builders on the command line.

sage_docbuild.ext.inventory_builder.**setup**(*app*)

2.2 Sage autodoc extension

This is sphinx.ext.autodoc extension modified for Sage objects.

The original header of sphinx.ext.autodoc:

Extension to create automatic documentation from code docstrings.

Automatically insert docstrings for functions, classes or whole modules into the doctree, thus avoiding duplication between docstrings and documentation for those who like elaborate docstrings.

This module is currently based on the legacy class-based implementation of `sphinx.ext.autodoc` from Sphinx version 9.1.0. Compare (do diff) with the upstream source file sphinx/ext/autodoc/_legacy_class_based/_documenters.py.

In the source file of this module, major modifications are delimited by a pair of comment dividers. To lessen maintenance burden, we aim at reducing the modifications. What Sphinx keeps outside that file – the directive options, the sentinels and the signature regular expressions – is imported from `sphinx.ext.autodoc`, which exports them publicly, so that a rebase only has to consider the parts that Sage really changes.

Note

Since Sphinx 9.1.0, the class-based documenters that this module derives from are the opt-in `autodoc_use_legacy_class_based` implementation: Sphinx builds its own `auto*` directives on a different implementation by default. Sage does not run into the two of them at once, because it loads this module in place of `sphinx.ext.autodoc`.

AUTHORS:

- ? (before 2011): initial version derived from sphinx.ext.autodoc
- Johan S. R. Nielsen: support for _sage_argspec_
- Simon King (2011-04): use sageinspect; include public cython attributes only in the documentation if they have a docstring
- Kwankyu Lee (2018-12-26, 2022-11-08): rebased on the latest sphinx.ext.autodoc
- Kwankyu Lee (2024-02-14): rebased on Sphinx 7.2.6
- François Bissey (2024-08-24): rebased on Sphinx 8.0.2
- François Bissey (2024-09-10): Tweaks to support python 3.9 (and older sphinx) as well
- François Bissey (2024-11-12): rebased on Sphinx 8.1.3 (while trying to keep python 3.9 compatibility)
- François Bissey (2025-02-24): Remove python 3.9 support hacks, making us closer to upstream
- François Bissey (2025-03-18): rebased on sphinx 8.2.3
- Chenxin Zhong (2026-07-07): rebased on Sphinx 9.1.0, where the class-based implementation this module derives from became `autodoc_use_legacy_class_based`
- Chenxin Zhong (2026-07-25): import the directive options and the sentinels from `sphinx.ext.autodoc` instead of copying them

class sage_docbuild.ext.sage_autodoc.**AttributeDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *GenericAliasMixin*, *SlotsMixin*, *RuntimeInstanceAttributeMixin*, *UninitializedInstanceAttributeMixin*, *NonDataDescriptorMixin*, *DocstringStripSignatureMixin*, *ClassLevelDocumenter*

Specialized Documenter subclass for attributes.

add_content(*more_content*)

add_directive_header(*sig*)

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

document_members(*all_members=False*)

get_attribute_comment(*parent*, *attrname*)

get_doc()

get_real_modname()

import_object(*raiseerror=False*)

static is_function_or_method(*obj*)

member_order = 60

order if autodoc_member_order is set to 'groupwise'

objtype = 'attribute'

name by which the directive is called (auto…) and the default generated directive name

option_spec: ClassVar[dict[str, Any]] = {'annotation': <function annotation_option>, 'no-index': <function bool_option>, 'no-index-entry': <function bool_option>, 'no-value': <function bool_option>, 'noindex': <function bool_option>}

priority = 10

priority if multiple documenters return True from can_document_member

should_suppress_value_header()

update_annotations(*parent*)

Update __annotations__ to support type_comment and so on.

class sage_docbuild.ext.sage_autodoc.**ClassDocumenter**(**args: Any*)

Bases: *DocstringSignatureMixin*, *ModuleLevelDocumenter*

Specialized Documenter subclass for classes.

add_content(*more_content*)

add_directive_header(*sig*)

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

document_members(*all_members=False*)

format_args(***kwargs*)

format_signature(***kwargs*)

generate(*more_content=None*, *real_modname=None*, *check_module=False*, *all_members=False*)

get_canonical_fullname()

get_doc()

get_object_members(*want_all*)

get_overloaded_signatures()

get_variable_comment()

import_object(*raiseerror=False*)

member_order = 20

order if autodoc_member_order is set to 'groupwise'

objtype = 'class'

name by which the directive is called (auto…) and the default generated directive name

option_spec: ClassVar[dict[str, Any]] = {'class-doc-from': <function class_doc_from_option>, 'exclude-members': <function exclude_members_option>, 'inherited-members': <function inherited_members_option>, 'member-order': <function member_order_option>, 'members': <function members_option>, 'no-index': <function bool_option>, 'no-index-entry': <function bool_option>, 'noindex': <function bool_option>, 'private-members': <function members_option>, 'show-inheritance': <function bool_option>, 'special-members': <function members_option>, 'undoc-members': <function bool_option>}

priority = 15

priority if multiple documenters return True from can_document_member

class sage_docbuild.ext.sage_autodoc.**ClassLevelDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *Documenter*

Specialized Documenter subclass for objects on class level (methods, attributes).

resolve_name(*modname*, *parents*, *path*, *base*)

class sage_docbuild.ext.sage_autodoc.**DataDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *GenericAliasMixin*, *UninitializedGlobalVariableMixin*, *ModuleLevelDocumenter*

Specialized Documenter subclass for data items.

add_content(*more_content*)

add_directive_header(*sig*)

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

document_members(*all_members=False*)

get_doc()

get_module_comment(*attrname*)

get_real_modname()

import_object(*raiseerror=False*)

member_order = 40

order if autodoc_member_order is set to 'groupwise'

objtype = 'data'

name by which the directive is called (auto…) and the default generated directive name

option_spec: ClassVar[dict[str, Any]] = {'annotation': <function annotation_option>, 'no-index': <function bool_option>, 'no-index-entry': <function bool_option>, 'no-value': <function bool_option>, 'noindex': <function bool_option>}

priority = -10

priority if multiple documenters return True from can_document_member

should_suppress_value_header()

update_annotations(*parent*)

Update __annotations__ to support type_comment and so on.

class sage_docbuild.ext.sage_autodoc.**DataDocumenterMixinBase**

Bases: *object*

config: Config

env: sphinx.environment.BuildEnvironment

modname: str

object: Any

objpath: list[str]

parent: Any

should_suppress_directive_header()

Check directive header should be suppressed.

should_suppress_value_header()

Check :value: header should be suppressed.

update_content(*more_content*)

Update docstring, for example with TypeVar variance.

class sage_docbuild.ext.sage_autodoc.**DecoratorDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *FunctionDocumenter*

Specialized Documenter subclass for decorator functions.

format_args(***kwargs*)

objtype = 'decorator'

name by which the directive is called (auto…) and the default generated directive name

priority = -1

priority if multiple documenters return True from can_document_member

class sage_docbuild.ext.sage_autodoc.**DocstringSignatureMixin**

Bases: object

Mixin for FunctionDocumenter and MethodDocumenter to provide the feature of reading the signature from the docstring.

format_signature(***kwargs*)

get_doc()

class sage_docbuild.ext.sage_autodoc.**DocstringStripSignatureMixin**

Bases: *DocstringSignatureMixin*

Mixin for AttributeDocumenter to provide the feature of stripping any function signature from the docstring.

format_signature(***kwargs*)

class sage_docbuild.ext.sage_autodoc.**Documenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: object

A Documenter knows how to autodocument a single object type. When registered with the AutoDirective, it will be used to document objects of that type when needed by autodoc.

Its *objtype* attribute selects what auto directive it is assigned to (the directive name is 'auto' + objtype), and what directive it generates by default, though that can be overridden by an attribute called *directivetype*.

A Documenter has an *option_spec* that works like a docutils directive's; in fact, it will be used to parse an auto directive's options that matches the Documenter.

add_content(*more_content*)

Add content from docstrings, attribute documentation and user.

add_directive_header(*sig*)

Add the directive header and options to the generated content.

add_line(*line*, *source*, **lineno*)

Append one line of generated reST to the output.

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

Called to see if a member can be documented by this Documenter.

check_module()

Check if *self.object* is really defined in the module given by *self.modname*.

content_indent = ' '

indentation by which to indent the directive content

document_members(*all_members=False*)

Generate reST for member documentation.

If *all_members* is True, document all members, else those given by *self.options.members*.

property documenters: dict[str, type[*Documenter*]]

Returns registered Documenter classes

filter_members(*members*, *want_all*)

Filter the given member list.

Members are skipped if

- they are private (except if given explicitly or the private-members option is set)
- they are special methods (except if given explicitly or the special-members option is set)
- they are undocumented (except if the undoc-members option is set)

The user can override the skipping decision by connecting to the `autodoc-skip-member` event.

format_args(***kwargs*)

Format the argument signature of *self.object*.

Should return None if the object does not have a signature.

format_name()

Format the name of *self.object*.

This normally should be something that can be parsed by the generated directive, but doesn't need to be (Sphinx will display it unparsed then).

format_signature(***kwargs*)

Format the signature (arguments and return annotation) of the object.

Let the user process it via the `autodoc-process-signature` event.

generate(*more_content=None*, *real_modname=None*, *check_module=False*, *all_members=False*)

Generate reST for the object given by *self.name*, and possibly for its members.

If *more_content* is given, include that content. If *real_modname* is given, use that module name to find attribute docs. If *check_module* is True, only generate if the object is defined in the module name it is imported from. If *all_members* is True, document all members.

get_attr(*obj*, *name*, **defargs*)

getattr() override for types such as Zope interfaces.

get_doc()

Decode and return lines of the docstring(s) for the object.

When it returns None, autodoc-process-docstring will not be called for this object.

get_object_members(*want_all*)

Return $(members_check_module, members)$ where $members$ is a list of $(membername, member)$ pairs of the members of *self.object*.

If *want_all* is True, return all members. Else, only return those members given by *self.options.members* (which may also be None).

get_real_modname()

Get the real module name of an object to document.

It can differ from the name of the module through which the object was imported.

get_sourcename()

import_object(*raiseerror=False*)

Import the object given by *self.modname* and *self.objpath* and set it as *self.object*.

Returns True if successful, False if an error occurred.

member_order = 0

order if autodoc_member_order is set to 'groupwise'

objtype = 'object'

name by which the directive is called (auto…) and the default generated directive name

option_spec: ClassVar[dict[str, Any]] = {'no-index': <function bool_option>, 'no-index-entry': <function bool_option>, 'noindex': <function bool_option>}

parse_name()

Determine what module to import and what attribute to document.

Returns True and sets *self.modname*, *self.objpath*, *self.fullname*, *self.args* and *self.retann* if parsing and resolving was successful.

priority = 0

priority if multiple documenters return True from can_document_member

process_doc(*docstrings*)

Let the user process the docstrings before adding them.

resolve_name(*modname*, *parents*, *path*, *base*)

Resolve the module and name of the object to document given by the arguments and the current module/class.

Must return a pair of the module name and a chain of attributes; for example, it would return `('zipfile', ['ZipFile', 'open'])` for the `zipfile.ZipFile.open` method.

sort_members(*documenters*, *order*)

Sort the given member list.

titles_allowed = True

true if the generated content may contain titles

class sage_docbuild.ext.sage_autodoc.**ExceptionDocumenter**(**args: Any*)

Bases: *ClassDocumenter*

Specialized ClassDocumenter subclass for exceptions.

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

member_order = 10

order if autodoc_member_order is set to 'groupwise'

objtype = 'exception'

name by which the directive is called (auto…) and the default generated directive name

priority = 20

priority if multiple documenters return True from can_document_member

class sage_docbuild.ext.sage_autodoc.**FunctionDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *DocstringSignatureMixin*, *ModuleLevelDocumenter*

Specialized Documenter subclass for functions.

add_directive_header(*sig*)

annotate_to_first_argument(*func*, *typ*)

Annotate type hint to the first argument of function if needed.

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

document_members(*all_members=False*)

format_args(***kwargs*)

format_signature(***kwargs*)

member_order = 30

order if autodoc_member_order is set to 'groupwise'

merge_default_value(*actual*, *overload*)

Merge default values of actual implementation to the overload variants.

objtype = 'function'

name by which the directive is called (auto…) and the default generated directive name

class sage_docbuild.ext.sage_autodoc.**GenericAliasMixin**

Bases: *DataDocumenterMixinBase*

Mixin for DataDocumenter and AttributeDocumenter to provide the feature for supporting GenericAliases.

should_suppress_directive_header()

update_content(*more_content*)

sage_docbuild.ext.sage_autodoc.**INSTANCEATTR = <object object>**

The base class of the class hierarchy.

When called, it accepts no arguments and returns a new featureless instance that has no instance attributes and cannot be given any.

class sage_docbuild.ext.sage_autodoc.**MethodDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *DocstringSignatureMixin*, *ClassLevelDocumenter*

Specialized Documenter subclass for methods (normal, static and class).

add_directive_header(*sig*)

annotate_to_first_argument(*func*, *typ*)

Annotate type hint to the first argument of function if needed.

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

directivetype = 'method'

document_members(*all_members=False*)

format_args(***kwargs*)

get_doc()

import_object(*raiseerror=False*)

member_order = 50

order if autodoc_member_order is set to 'groupwise'

merge_default_value(*actual*, *overload*)

Merge default values of actual implementation to the overload variants.

objtype = 'method'

name by which the directive is called (auto…) and the default generated directive name

priority = 1

priority if multiple documenters return True from can_document_member

class sage_docbuild.ext.sage_autodoc.**ModuleDocumenter**(**args: Any*)

Bases: *Documenter*

Specialized Documenter subclass for modules.

add_content(*more_content*)

add_directive_header(*sig*)

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

content_indent = ''

indentation by which to indent the directive content

get_module_members()

Get members of target module.

get_object_members(*want_all*)

import_object(*raiseerror=False*)

objtype = 'module'

name by which the directive is called (auto…) and the default generated directive name

option_spec: ClassVar[dict[str, Any]] = {'deprecated': <function bool_option>, 'exclude-members': <function exclude_members_option>, 'ignore-module-all': <function bool_option>, 'imported-members': <function bool_option>, 'inherited-members': <function inherited_members_option>, 'member-order': <function member_order_option>, 'members': <function members_option>, 'no-index': <function bool_option>, 'no-index-entry': <function bool_option>, 'no-value': <function bool_option>, 'noindex': <function bool_option>, 'platform': <function identity>, 'private-members': <function members_option>, 'show-inheritance': <function bool_option>, 'special-members': <function members_option>, 'synopsis': <function identity>, 'undoc-members': <function bool_option>}

parse_name()

resolve_name(*modname*, *parents*, *path*, *base*)

sort_members(*documenters*, *order*)

class sage_docbuild.ext.sage_autodoc.**ModuleLevelDocumenter**(*directive: Any*, *name: str*, *indent: str* = *''*)

Bases: *Documenter*

Specialized Documenter subclass for objects on module level (functions, classes, data/constants).

resolve_name(*modname*, *parents*, *path*, *base*)

class sage_docbuild.ext.sage_autodoc.**NonDataDescriptorMixin**

Bases: *DataDocumenterMixinBase*

Mixin for AttributeDocumenter to provide the feature for supporting non data-descriptors.

Note

This mix-in must be inherited after other mix-ins. Otherwise, docstring and :value: header will be suppressed unexpectedly.

get_doc()

import_object(*raiseerror=False*)

should_suppress_value_header()

class sage_docbuild.ext.sage_autodoc.**ObjectMember**(*name: str*, *obj: Any*, ***, *docstring: str | None* = *None*, *class_: Any* = *None*, *skipped: bool* = *False*)

Bases: *object*

A member of object.

This is used for the result of $Documenter.get_{m}odule_{m}embers()$ to represent each member of the object.

class_: Any

docstring: str | None

object: Any

skipped: bool

class sage_docbuild.ext.sage_autodoc.**PropertyDocumenter**(*directive: Any*, *name: str*, *indent: str = ''*)

Bases: *DocstringStripSignatureMixin*, *ClassLevelDocumenter*

Specialized Documenter subclass for properties.

add_directive_header(*sig*)

classmethod can_document_member(*member*, *membername*, *isattr*, *parent*)

document_members(*all_members=False*)

format_args(***kwargs*)

get_real_modname()

import_object(*raiseerror=False*)

Check the existence of uninitialized instance attribute when failed to import the attribute.

member_order = 60

order if autodoc_member_order is set to 'groupwise'

objtype = 'property'

name by which the directive is called (auto…) and the default generated directive name

priority = 11

priority if multiple documenters return True from can_document_member

class sage_docbuild.ext.sage_autodoc.**RuntimeInstanceAttributeMixin**

Bases: *DataDocumenterMixinBase*

Mixin for AttributeDocumenter to provide the feature for supporting runtime instance attributes (that are defined in __init__() methods with doc-comments).

Example:

```
class Foo:
    def __init__(self):
        self.attr = None  #: This is a target of this mix-in.
```

RUNTIME_INSTANCE_ATTRIBUTE

get_doc()

import_object(*raiseerror=False*)

Check the existence of runtime instance attribute after failing to import the attribute.

is_runtime_instance_attribute(*parent*)

Check the subject is an attribute defined in __init__().

is_runtime_instance_attribute_not_commented(*parent*)

Check the subject is an attribute defined in __init__() without comment.

should_suppress_value_header()

class sage_docbuild.ext.sage_autodoc.**SlotsMixin**

Bases: *DataDocumenterMixinBase*

Mixin for AttributeDocumenter to provide the feature for supporting __slots__.

get_doc()

import_object(*raiseerror=False*)

isslotsattribute()

Check the subject is an attribute in __slots__.

should_suppress_value_header()

class sage_docbuild.ext.sage_autodoc.**UninitializedGlobalVariableMixin**

Bases: *DataDocumenterMixinBase*

Mixin for DataDocumenter to provide the feature for supporting uninitialized (type annotation only) global variables.

get_doc()

import_object(*raiseerror=False*)

should_suppress_value_header()

class sage_docbuild.ext.sage_autodoc.**UninitializedInstanceAttributeMixin**

Bases: *DataDocumenterMixinBase*

Mixin for AttributeDocumenter to provide the feature for supporting uninitialized instance attributes (PEP-526 styled, annotation only attributes).

Example:

```
class Foo:
    attr: int  #: This is a target of this mix-in.
```

get_doc()

import_object(*raiseerror=False*)

Check the existence of uninitialized instance attribute when failed to import the attribute.

is_uninitialized_instance_attribute(*parent*)

Check the subject is an annotation only attribute.

should_suppress_value_header()

sage_docbuild.ext.sage_autodoc.**autodoc_attrgetter**(*obj*, *name*, *registry*, **defargs*)

Alternative getattr() for types

sage_docbuild.ext.sage_autodoc.**getdoc**(*obj*, **args*, ***kwargs*)

Read a docstring the way Sage does, in place of Sphinx's own reader.

This accepts the calling convention of `sphinx.util.inspect.getdoc` rather than repeating its signature, so that the documenters below can call it either way round. Everything past the object is discarded: Sage resolves inheritance and Cython docstrings itself, so the `attrgetter`, `allow_inherited`, `cls` and `name` that Sphinx would use have nothing left to decide.

sage_docbuild.ext.sage_autodoc.**restify**(*cls*, *mode='fully-qualified-except-typing'*)

Convert a type-like object to a reST reference.

sage_docbuild.ext.sage_autodoc.**setup**(*app*)

sage_docbuild.ext.sage_autodoc.**stringify_annotation**(*annotation*, *mode*, *short_literals='fully-qualified-except-typing'*)

Stringify a type annotation.

sage_docbuild.ext.sage_autodoc.**stringify_signature**(*sig*, *show_annotation=True*, *show_return_annotation=True*, *unqualified_typehints=False*, *short_literals=False*)

Stringify a signature.

2.3 Sage multidocs extension

The goal of this extension is to manage a multi-documentation in Sphinx. To be able to compile Sage's huge documentation in parallel, the documentation is cut into a bunch of independent documentations called "sub-docs", which are compiled separately. There is a master document which points to all the sub-docs. The intersphinx extension ensures that the cross-link between the sub-docs are correctly resolved. However some work is needed to build a global index. This is the goal of the `multidocs` extension.

More precisely this extension ensures the correct merging of

- the todo list if this extension is activated
- the python indexes
- the list of python modules
- the javascript index
- the citations

sage_docbuild.ext.multidocs.**citation_dir**(*app*)

sage_docbuild.ext.multidocs.**fetch_citation**(*app*, *env*)

Fetch the global citation index from the refman to allow for cross references.

sage_docbuild.ext.multidocs.**fix_path_html**(*app*, *pagename*, *templatename*, *ctx*, *event_arg*)

Fix the context so that the files

- `search.html`
- `genindex.html`
- `py-modindex.html`

point to the right place, that is in `reference/` instead of `reference/subdocument`.

sage_docbuild.ext.multidocs.**get_env**(*app*, *curdoc*)

Get the environment of a sub-doc from the pickle

sage_docbuild.ext.multidocs.**get_js_index**(*app*, *curdoc*)

Get the JS index of a sub-doc from the file

sage_docbuild.ext.multidocs.**init_subdoc**(*app*)

Init the merger depending on if we are compiling a sub-doc or the master doc itself.

sage_docbuild.ext.multidocs.**merge_environment**(*app*, *env*)

Merge the following attributes of the sub-docs environment into the main environment:

- `titles` – Titles

- `todo_all_todos` – todo's
- `indexentries` – global python index
- `all_docs` – needed by the js index
- `citations` – citations
- `domaindata['py']['modules']` – list of python modules

`sage_docbuild.ext.multidocs.`**`merge_js_index`**(*app*)

Merge the JS indexes of the sub-docs into the main JS index

`sage_docbuild.ext.multidocs.`**`setup`**(*app*)

`sage_docbuild.ext.multidocs.`**`write_citations`**(*app*, *citations*)

Pickle the citation in a file.

2.4 Cross-references between Sage and other projects

This extension configures intersphinx for the documentation of the projects that Sage refers to, and resolves the references that Sphinx alone cannot, either because the target is documented under a public name other than the one that the reference names, or because it has no target to link to.

`sage_docbuild.ext.crossrefs.`**`call_intersphinx`**(*app*, *env*, *node*, *contnode*)

Call intersphinx and make links between Sage manuals relative.

`sage_docbuild.ext.crossrefs.`**`debug_inf`**(*app*, *message*)

`sage_docbuild.ext.crossrefs.`**`find_sage_dangling_links`**(*app*, *env*, *node*, *contnode*)

Try to find dangling link in local module imports or all.py.

`sage_docbuild.ext.crossrefs.`**`prefer_python_inventory`**(*app*)

Give the Python inventory the last word on the builtins it shares.

Intersphinx merges every inventory into a single one, project by project in the order of the project names, so that the project whose name sorts last silently claims each name that two projects define. Sphinx documents `enumerate` as a function of its own in the quickstart guide, for one, which is enough to divert every reference to the builtin away from the Python manual.

Only the builtins are taken back, and only in the Python domain: a name that a project legitimately owns keeps its target, be it a glossary term, a document, a label or a function of the C API that the Python manual happens to name too.

`sage_docbuild.ext.crossrefs.`**`set_intersphinx_mappings`**(*app*, *config*)

Add precompiled inventory (the objects.inv)

`sage_docbuild.ext.crossrefs.`**`setup`**(*app*)

Register this extension with Sphinx.

2.5 Live code blocks

With `SAGE_LIVE_DOC=yes`, this extension turns the examples of the documentation into code blocks that the reader can execute, using jupyter-sphinx. Otherwise it makes the jupyter-sphinx directives no-ops.

`class` `sage_docbuild.ext.livedoc.`**`Ignore`**(*name*, *arguments*, *options*, *content*, *lineno*, *content_offset*, *block_text*, *state*, *state_machine*)

Bases: `SphinxDirective`

has_content = True

May the directive have content?

run()

class sage_docbuild.ext.livedoc.**SagecodeTransform**(*document*, *startnode=None*)

Bases: SphinxTransform

Transform a code block to a live code block enabled by jupyter-sphinx.

Effectively a code block like:

```
EXAMPLE::

    sage: 1 + 1
    2
```

is transformed into:

```
EXAMPLE::

    sage: 1 + 1
    2

.. ONLY:: html

    .. JUPYTER-EXECUTE::
        :hide-code:
        :hide-output:
        :raises:
        :stderr:

        1 + 1
```

enabling live execution of the code.

apply()

default_priority = 170

Numerical priority of this transform, 0 through 999 (override).

sage_docbuild.ext.livedoc.**ignore_jupyter_directives**(*app*)

Make the directives of jupyter-sphinx do nothing.

Sphinx sets an extension up in the order of `extensions`, where jupyter-sphinx comes after this one, so registering the directives here would leave the ones of jupyter-sphinx in place; they are registered once every extension is set up instead. Executing the examples then needs a Sage kernel, which a documentation build has no reason to require.

sage_docbuild.ext.livedoc.**setup**(*app*)

Register this extension with Sphinx.

2.6 Selection of the members to document

This extension decides which members Sphinx includes in the reference manual, and optionally reports the nested classes whose pickling is broken.

sage_docbuild.ext.members.**check_nested_class_picklability**(*app*, *what*, *name*, *obj*, *skip*, *options*)

Print a warning if pickling is broken for nested classes.

sage_docbuild.ext.members.**setup**(*app*)

Register this extension with Sphinx.

sage_docbuild.ext.members.**skip_member**(*app*, *what*, *name*, *obj*, *skip*, *options*)

To suppress Sphinx warnings / errors, we

- Don't include [aliases of] builtins.
- Don't include the docstring for any nested class which has been inserted into its module by sage.misc.nested_class.NestedClassMetaclass only for pickling. The class will be properly documented inside its surrounding class.
- Optionally, check whether pickling is broken for nested classes.
- Optionally, include objects whose name begins with an underscore ('_'), i.e., "private" or "hidden" attributes, methods, etc.

Otherwise, we abide by Sphinx's decision. Note: The object obj is excluded (included) if this handler returns True (False).

PYTHON MODULE INDEX

S

sage_docbuild.__main__, 1
sage_docbuild.build_options, 18
sage_docbuild.builders, 4
sage_docbuild.conf, 22
sage_docbuild.ext.crossrefs, 42
sage_docbuild.ext.inventory_builder, 29
sage_docbuild.ext.livedoc, 42
sage_docbuild.ext.members, 43
sage_docbuild.ext.multidocs, 41
sage_docbuild.ext.sage_autodoc, 29
sage_docbuild.sphinxbuild, 19
sage_docbuild.utils, 24

INDEX

A

add_content() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 30
add_content() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
add_content() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
add_content() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
add_content() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 37
add_directive_header() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 30
add_directive_header() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
add_directive_header() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
add_directive_header() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
add_directive_header() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
add_directive_header() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
add_directive_header() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 37
add_directive_header() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter method*), 39
add_line() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
add_page_context() (*in module sage_docbuild.conf*), 22
annotate_to_first_argument() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
annotate_to_first_argument() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
ansi_escape_sequence (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
ansi_escape_sequence_color (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
apply() (*sage_docbuild.ext.livedoc.SagecodeTransform method*), 43
AttributeDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 30
auto_rest_filename() (*sage_docbuild.builders.ReferenceSubBuilder method*), 6
autodoc_attrgetter() (*in module sage_docbuild.ext.sage_autodoc*), 40

B

build_many() (*in module sage_docbuild.builders*), 8
build_many() (*in module sage_docbuild.utils*), 24
build_ref_doc() (*in module sage_docbuild.builders*), 8
builder_helper() (*in module sage_docbuild.builders*), 8
BuildOptions (*class in sage_docbuild.build_options*), 18

C

cache_file() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
call_intersphinx() (*in module sage_docbuild.ext.crossrefs*), 42
can_document_member() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter class method*), 30
can_document_member() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter class method*), 31
can_document_member() (*sage_docbuild.ext.sage_autodoc.DataDocumenter class method*), 32
can_document_member() (*sage_docbuild.ext.sage_autodoc.Documenter*

class method), 34
can_document_member() (*sage_docbuild.ext.sage_autodoc.ExceptionDocumenter class method*), 36
can_document_member() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter class method*), 36
can_document_member() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter class method*), 37
can_document_member() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter class method*), 37
can_document_member() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter class method*), 39
changes() (*sage_docbuild.builders.DocBuilder method*), 5
check_module() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
check_nested_class_picklability() (*in module sage_docbuild.ext.members*), 43
citation_dir() (*in module sage_docbuild.ext.multidocs*), 41
citation_labels() (*in module sage_docbuild.sphinxbuild*), 21
class_ (*sage_docbuild.ext.sage_autodoc.ObjectMember attribute*), 38
ClassDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 31
ClassLevelDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 32
clean() (*sage_docbuild.builders.DocBuilder method*), 5
clean() (*sage_docbuild.builders.WebsiteBuilder method*), 8
clean_auto() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
close() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 20
closed (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
command_line_args() (*in module sage_docbuild.__main__*), 2
config (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
content_indent (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 34
content_indent (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter attribute*), 37

D

DataDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 32
DataDocumenterMixinBase (*class in sage_docbuild.ext.sage_autodoc*), 33
debug_inf() (*in module sage_docbuild.ext.crossrefs*), 42
DecoratorDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 33
default_priority (*sage_docbuild.ext.livedoc.SageCodeTransform attribute*), 43
directivetype (*sage_docbuild.ext.sage_autodoc.MethodDocumenter attribute*), 37
DocBuilder (*class in sage_docbuild.builders*), 5
docstring (*sage_docbuild.ext.sage_autodoc.ObjectMember attribute*), 38
DocstringSignatureMixin (*class in sage_docbuild.ext.sage_autodoc*), 33
DocstringStripSignatureMixin (*class in sage_docbuild.ext.sage_autodoc*), 33
document_members() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 30
document_members() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
document_members() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
document_members() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
document_members() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
document_members() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
document_members() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter method*), 39
Documenter (*class in sage_docbuild.ext.sage_autodoc*), 33
documenters (*sage_docbuild.ext.sage_autodoc.Documenter property*), 34
documents_single_file (*sage_docbuild.builders.DocBuilder attribute*), 5
documents_single_file (*sage_docbuild.builders.SingleFileBuilder attribute*), 8

E

encoding (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
env (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
environment variable
SAGE_DOCBUILD_OPTS, 2, 4
epilog (*sage_docbuild.ext.inventory_builder.InventoryBuilder attribute*), 29
exc (*sage_docbuild.utils.RemoteExceptionWrapper attribute*), 24
ExceptionDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 36

F

fetch_citation() (*in module sage_docbuild.ext.multi-*

docs), 41
fetch_inventory() (*sage_docbuild.__main__.InterSphinxCache method*), 2
filter_members() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
find_sage_dangling_links() (*in module sage_docbuild.ext.crossrefs*), 42
finish() (*sage_docbuild.ext.inventory_builder.InventoryBuilder method*), 29
fix_path_html() (*in module sage_docbuild.ext.multidocs*), 41
flush() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 20
format (*sage_docbuild.ext.inventory_builder.InventoryBuilder attribute*), 29
format_args() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
format_args() (*sage_docbuild.ext.sage_autodoc.DecoratorDocumenter method*), 33
format_args() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
format_args() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
format_args() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
format_args() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter method*), 39
format_columns() (*in module sage_docbuild.__main__*), 2
format_name() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
format_signature() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
format_signature() (*sage_docbuild.ext.sage_autodoc.DocstringSignatureMixin method*), 33
format_signature() (*sage_docbuild.ext.sage_autodoc.DocstringStripSignatureMixin method*), 33
format_signature() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 34
format_signature() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
FunctionDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 36

G

generate() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
generate() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
generate_doc_sources() (*in module sage_docbuild.__main__*), 2
GenericAliasMixin (*class in sage_docbuild.ext.sage_autodoc*), 36
get_all_documents() (*in module sage_docbuild.builders*), 9
get_all_included_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_all_reference_documents() (*in module sage_docbuild.builders*), 9
get_all_rst_files() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_attr() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
get_attribute_comment() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 30
get_builder() (*in module sage_docbuild.builders*), 10
get_cache() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_canonical_fullname() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
get_doc() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 30
get_doc() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
get_doc() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
get_doc() (*sage_docbuild.ext.sage_autodoc.DocstringSignatureMixin method*), 33
get_doc() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
get_doc() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
get_doc() (*sage_docbuild.ext.sage_autodoc.NonDataDescriptorMixin method*), 38
get_doc() (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin method*), 39
get_doc() (*sage_docbuild.ext.sage_autodoc.SlotsMixin method*), 40
get_doc() (*sage_docbuild.ext.sage_autodoc.UninitializedGlobalVariableMixin method*), 40
get_doc() (*sage_docbuild.ext.sage_autodoc.UninitializedInstanceAttributeMixin method*), 40
get_documents() (*in module sage_docbuild.builders*), 11
get_env() (*in module sage_docbuild.ext.multidocs*), 41
get_formats() (*in module sage_docbuild.__main__*), 3
get_js_index() (*in module sage_docbuild.ext.multidocs*), 41
get_modified_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_module_comment() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32

get_module_docstring_title() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_module_members() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 37
get_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_new_and_updated_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_object_members() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
get_object_members() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
get_object_members() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 37
get_outdated_docs() (*sage_docbuild.ext.inventory_builder.InventoryBuilder method*), 29
get_overloaded_signatures() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
get_real_modname() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 31
get_real_modname() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
get_real_modname() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
get_real_modname() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter method*), 39
get_sourcename() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
get_sphinx_environment() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_target_uri() (*sage_docbuild.ext.inventory_builder.InventoryBuilder method*), 29
get_unincluded_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
get_variable_comment() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
getdoc() (*in module sage_docbuild.ext.sage_autodoc*), 40

H

has_content (*sage_docbuild.ext.livedoc.Ignore attribute*), 42
help_commands() (*in module sage_docbuild.__main__*), 3
help_description() (*in module sage_docbuild.__main__*), 3
help_documents() (*in module sage_docbuild.__main__*), 3
help_examples() (*in module sage_docbuild.__main__*), 3
help_formats() (*in module sage_docbuild.__main__*), 3
help_message_long (*class in sage_docbuild.__main__*), 3
help_message_short (*class in sage_docbuild.__main__*), 3
help_usage() (*in module sage_docbuild.__main__*), 3
help_wrapper (*class in sage_docbuild.__main__*), 3
html() (*sage_docbuild.builders.DocBuilder method*), 5
html() (*sage_docbuild.builders.ReferenceTopBuilder method*), 8
html() (*sage_docbuild.builders.WebsiteBuilder method*), 8
htmlhelp() (*sage_docbuild.builders.DocBuilder method*), 5

I

Ignore (*class in sage_docbuild.ext.livedoc*), 42
ignore_jupyter_directives() (*in module sage_docbuild.ext.livedoc*), 43
import_object() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 31
import_object() (*sage_docbuild.ext.sage_autodoc.ClassDocumenter method*), 31
import_object() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
import_object() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
import_object() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
import_object() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 38
import_object() (*sage_docbuild.ext.sage_autodoc.NonDataDescriptorMixin method*), 38
import_object() (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter method*), 39
import_object() (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin method*), 39
import_object() (*sage_docbuild.ext.sage_autodoc.SlotsMixin method*), 40
import_object() (*sage_docbuild.ext.sage_autodoc.UninitializedGlobalVariableMixin method*), 40
import_object() (*sage_docbuild.ext.sage_autodoc.UninitializedInstanceAttributeMixin method*), 40
init_subdoc() (*in module sage_docbuild.ext.multidocs*), 41
INSTANCEATTR (*in module sage_docbuild.ext.sage_autodoc*), 37
IntersphinxCache (*class in sage_docbuild.__main__*), 2

inventory() (*sage_docbuild.builders.DocBuilder method*), 5
InventoryBuilder (*class in sage_docbuild.ext.inventory_builder*), 29
is_function_or_method() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter static method*), 31
is_runtime_instance_attribute() (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin method*), 39
is_runtime_instance_attribute_not_commented() (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin method*), 39
is_uninitialized_instance_attribute() (*sage_docbuild.ext.sage_autodoc.UninitializedInstanceAttributeMixin method*), 40
isatty() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 20
isslotsattribute() (*sage_docbuild.ext.sage_autodoc.SlotsMixin method*), 40

J

json() (*sage_docbuild.builders.DocBuilder method*), 5

K

KnownCitation (*class in sage_docbuild.sphinxbuild*), 19

L

latex() (*sage_docbuild.builders.DocBuilder method*), 5
linkcheck() (*sage_docbuild.builders.DocBuilder method*), 5
linkcode_resolve() (*in module sage_docbuild.conf*), 22
load_single_file() (*in module sage_docbuild.builders*), 12

M

main() (*in module sage_docbuild.__main__*), 4
member_order (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter attribute*), 31
member_order (*sage_docbuild.ext.sage_autodoc.ClassDocumenter attribute*), 31
member_order (*sage_docbuild.ext.sage_autodoc.DataDocumenter attribute*), 32
member_order (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 35
member_order (*sage_docbuild.ext.sage_autodoc.ExceptionDocumenter attribute*), 36
member_order (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter attribute*), 36
member_order (*sage_docbuild.ext.sage_autodoc.MethodDocumenter attribute*), 37
member_order (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter attribute*), 39
merge_default_value() (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter method*), 36
merge_default_value() (*sage_docbuild.ext.sage_autodoc.MethodDocumenter method*), 37
merge_environment() (*in module sage_docbuild.ext.multidocs*), 41
merge_js_index() (*in module sage_docbuild.ext.multidocs*), 42
MethodDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 37
mode (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
modname (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
module
    sage_docbuild.__main__, 1
    sage_docbuild.build_options, 18
    sage_docbuild.builders, 4
    sage_docbuild.conf, 22
    sage_docbuild.ext.crossrefs, 42
    sage_docbuild.ext.inventory_builder, 29
    sage_docbuild.ext.livedoc, 42
    sage_docbuild.ext.members, 43
    sage_docbuild.ext.multidocs, 41
    sage_docbuild.ext.sage_autodoc, 29
    sage_docbuild.sphinxbuild, 19
    sage_docbuild.utils, 24
ModuleDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 37
ModuleLevelDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 38

N

name (*sage_docbuild.ext.inventory_builder.InventoryBuilder attribute*), 29
name (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
newlines (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
NonDataDescriptorMixin (*class in sage_docbuild.ext.sage_autodoc*), 38

O

object (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
object (*sage_docbuild.ext.sage_autodoc.ObjectMember attribute*), 39
ObjectMember (*class in sage_docbuild.ext.sage_autodoc*), 38

objpath (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
objtype (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter attribute*), 31
objtype (*sage_docbuild.ext.sage_autodoc.ClassDocumenter attribute*), 31
objtype (*sage_docbuild.ext.sage_autodoc.DataDocumenter attribute*), 32
objtype (*sage_docbuild.ext.sage_autodoc.DecoratorDocumenter attribute*), 33
objtype (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 35
objtype (*sage_docbuild.ext.sage_autodoc.ExceptionDocumenter attribute*), 36
objtype (*sage_docbuild.ext.sage_autodoc.FunctionDocumenter attribute*), 36
objtype (*sage_docbuild.ext.sage_autodoc.MethodDocumenter attribute*), 37
objtype (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter attribute*), 38
objtype (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter attribute*), 39
option_spec (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter attribute*), 31
option_spec (*sage_docbuild.ext.sage_autodoc.ClassDocumenter attribute*), 32
option_spec (*sage_docbuild.ext.sage_autodoc.DataDocumenter attribute*), 32
option_spec (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 35
option_spec (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter attribute*), 38
original_exception (*sage_docbuild.utils.WorkerDiedException attribute*), 24
output_dir (*sage_docbuild.build_options.BuildOptions attribute*), 18
output_dir_given (*sage_docbuild.build_options.BuildOptions attribute*), 18

P

parent (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase attribute*), 33
parse_name() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
parse_name() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 38
pdf() (*sage_docbuild.builders.DocBuilder method*), 5
pdf() (*sage_docbuild.builders.SingleFileBuilder method*), 8
pdf() (*sage_docbuild.builders.WebsiteBuilder method*), 8
pickle() (*sage_docbuild.builders.DocBuilder method*), 6
prefer_python_inventory() (*in module sage_docbuild.ext.crossrefs*), 42
prefix_len (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
print_included_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
print_modified_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
print_new_and_updated_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
print_unincluded_modules() (*sage_docbuild.builders.ReferenceBuilder method*), 6
print_unincluded_modules() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
priority (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter attribute*), 31
priority (*sage_docbuild.ext.sage_autodoc.ClassDocumenter attribute*), 32
priority (*sage_docbuild.ext.sage_autodoc.DataDocumenter attribute*), 32
priority (*sage_docbuild.ext.sage_autodoc.DecoratorDocumenter attribute*), 33
priority (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 35
priority (*sage_docbuild.ext.sage_autodoc.ExceptionDocumenter attribute*), 36
priority (*sage_docbuild.ext.sage_autodoc.MethodDocumenter attribute*), 37
priority (*sage_docbuild.ext.sage_autodoc.PropertyDocumenter attribute*), 39
process_doc() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
PropertyDocumenter (*class in sage_docbuild.ext.sage_autodoc*), 39

R

raise_errors() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 20
reference_subdocument() (*in module sage_docbuild.conf*), 22
ReferenceBuilder (*class in sage_docbuild.builders*), 6
ReferenceSubBuilder (*class in sage_docbuild.builders*), 6
ReferenceTopBuilder (*class in sage_docbuild.builders*), 8
RemoteException, 24
RemoteExceptionWrapper (*class in sage_docbuild.utils*), 24
resolve_name() (*sage_docbuild.ext.sage_autodoc.ClassLevelDocumenter method*), 32

resolve_name() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 35
resolve_name() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 38
resolve_name() (*sage_docbuild.ext.sage_autodoc.ModuleLevelDocumenter method*), 38
restify() (*in module sage_docbuild.ext.sage_autodoc*), 40
run() (*sage_docbuild.ext.livedoc.Ignore method*), 43
runsphinx() (*in module sage_docbuild.sphinxbuild*), 21
RUNTIME_INSTANCE_ATTRIBUTE (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin attribute*), 39
RuntimeInstanceAttributeMixin (*class in sage_docbuild.ext.sage_autodoc*), 39

S

sage_docbuild.__main__
    module, 1
SAGE_DOCBUILD_OPTS, 2, 4
sage_docbuild.build_options
    module, 18
sage_docbuild.builders
    module, 4
sage_docbuild.conf
    module, 22
sage_docbuild.ext.crossrefs
    module, 42
sage_docbuild.ext.inventory_builder
    module, 29
sage_docbuild.ext.livedoc
    module, 42
sage_docbuild.ext.members
    module, 43
sage_docbuild.ext.multidocs
    module, 41
sage_docbuild.ext.sage_autodoc
    module, 29
sage_docbuild.sphinxbuild
    module, 19
sage_docbuild.utils
    module, 24
sage_root_for() (*in module sage_docbuild.__main__*), 4
SagecodeTransform (*class in sage_docbuild.ext.livedoc*), 43
SageSphinxLogger (*class in sage_docbuild.sphinxbuild*), 19
save_cache() (*sage_docbuild.builders.ReferenceSubBuilder method*), 7
search() (*sage_docbuild.sphinxbuild.KnownCitation method*), 19
set_intersphinx_mappings() (*in module sage_docbuild.ext.crossrefs*), 42
setup() (*in module sage_docbuild.conf*), 23
setup() (*in module sage_docbuild.ext.crossrefs*), 42
setup() (*in module sage_docbuild.ext.inventory_builder*), 29
setup() (*in module sage_docbuild.ext.livedoc*), 43
setup() (*in module sage_docbuild.ext.members*), 44
setup() (*in module sage_docbuild.ext.multidocs*), 42
setup() (*in module sage_docbuild.ext.sage_autodoc*), 40
setup_logger() (*in module sage_docbuild.__main__*), 4
setup_parser() (*in module sage_docbuild.__main__*), 4
should_suppress_directive_header() (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase method*), 33
should_suppress_directive_header() (*sage_docbuild.ext.sage_autodoc.GenericAliasMixin method*), 36
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 31
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase method*), 33
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.NonDataDescriptorMixin method*), 38
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.RuntimeInstanceAttributeMixin method*), 39
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.SlotsMixin method*), 40
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.UninitializedGlobalVariableMixin method*), 40
should_suppress_value_header() (*sage_docbuild.ext.sage_autodoc.UninitializedInstanceAttributeMixin method*), 40
single_file_ignored_warnings() (*in module sage_docbuild.sphinxbuild*), 21
SingleFileBuilder (*class in sage_docbuild.builders*), 8
skip_member() (*in module sage_docbuild.ext.members*), 44
skipped (*sage_docbuild.ext.sage_autodoc.ObjectMember attribute*), 39
SlotsMixin (*class in sage_docbuild.ext.sage_autodoc*), 39
softspace (*sage_docbuild.sphinxbuild.SageSphinxLogger attribute*), 20
sort_members() (*sage_docbuild.ext.sage_autodoc.Documenter method*), 36
sort_members() (*sage_docbuild.ext.sage_autodoc.ModuleDocumenter method*), 38

source_dir (*sage_docbuild.build_options.BuildOptions attribute*), 18
source_dir_for_help() (*in module sage_docbuild.__main__*), 4
stringify_annotation() (*in module sage_docbuild.ext.sage_autodoc*), 41
stringify_signature() (*in module sage_docbuild.ext.sage_autodoc*), 41

T

tb (*sage_docbuild.utils.RemoteException attribute*), 24
tb (*sage_docbuild.utils.RemoteExceptionWrapper attribute*), 24
term_width_line() (*in module sage_docbuild.sphinxbuild*), 22
titles_allowed (*sage_docbuild.ext.sage_autodoc.Documenter attribute*), 36

U

UninitializedGlobalVariableMixin (*class in sage_docbuild.ext.sage_autodoc*), 40
UninitializedInstanceAttributeMixin (*class in sage_docbuild.ext.sage_autodoc*), 40
update_annotations() (*sage_docbuild.ext.sage_autodoc.AttributeDocumenter method*), 31
update_annotations() (*sage_docbuild.ext.sage_autodoc.DataDocumenter method*), 32
update_content() (*sage_docbuild.ext.sage_autodoc.DataDocumenterMixinBase method*), 33
update_content() (*sage_docbuild.ext.sage_autodoc.GenericAliasMixin method*), 37
update_mtimes() (*sage_docbuild.builders.ReferenceSubBuilder method*), 8

W

web() (*sage_docbuild.builders.DocBuilder method*), 6
WebsiteBuilder (*class in sage_docbuild.builders*), 8
WorkerDiedException, 24
write() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 21
write_auto_rest_file() (*sage_docbuild.builders.ReferenceSubBuilder method*), 8
write_citations() (*in module sage_docbuild.ext.multidocs*), 42
writelines() (*sage_docbuild.sphinxbuild.SageSphinxLogger method*), 21